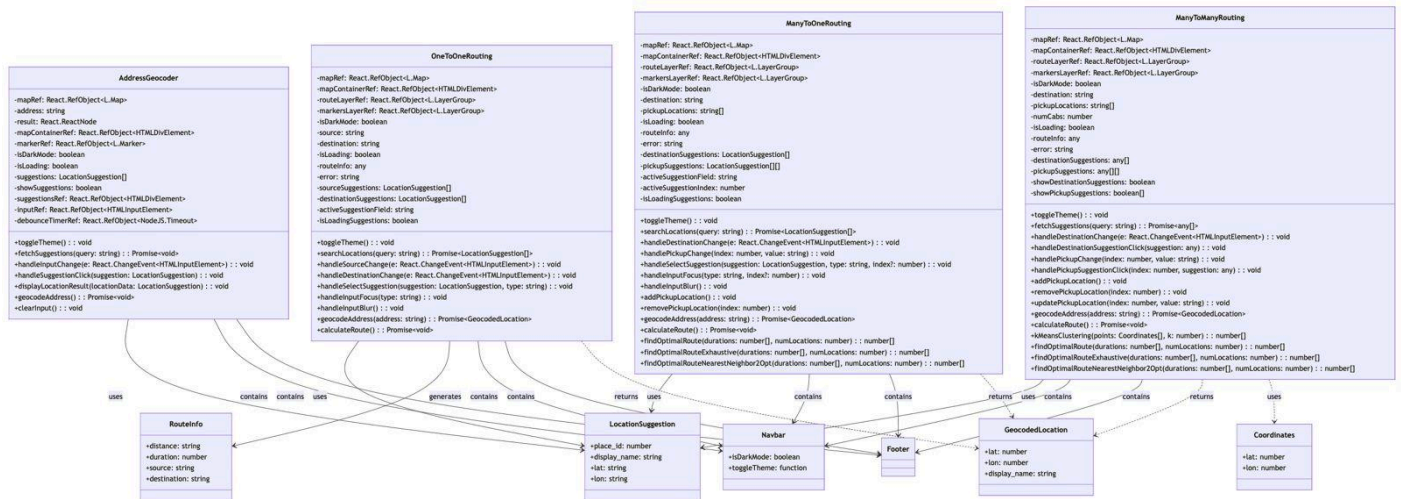


Product Design Document

Team Fleetix (Team No. 28) [Akshitha, Aryanil, Aviral, Satvik, Sherley]

Note: This is a “living document”, meaning its content and format will grow with the implementation of the project.

Design Model



(apologies for the sizing ratio)

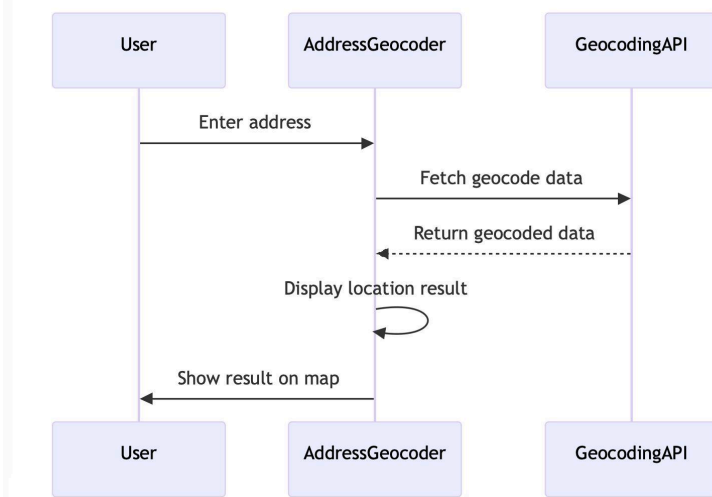
Geocoder	Class State • No explicit persistent state (it relies on external configurations and libraries for geocoding). Class Behavior • geocode(address: String): Coordinates – Converts an address string into geographic coordinates. • validateAddress(address: String): Boolean – Checks if the provided address is valid.
Employee	Class State • employeeId: String – Unique identifier for the employee. • name: String – Employee's name. • address: String – Employee's address. • coordinates: Coordinates – Geographic coordinates obtained via geocoding. • assignedRoute: Route – The route assigned to the employee by the RoutingEngine. Class Behavior • updatePreferences(preferences: Preferences): void – Updates the employee's pickup and drop-off preferences.
RoadNetwork	Class State • networkGraph: Graph – Represents the city/town's road network as a graph structure. Class behavior • loadNetwork(): Graph – Loads and structures the road network data. • getPath(src: Coordinates, dest: Coordinates): Route – Computes a route between source and destination coordinates.
RoutingEngine	Class State • No persistent state (it processes input parameters on demand). Class Behavior • calculateRoute(employees: List<Employee>, maxDelay: int, waitTime: int): Route – Computes an optimal route for the given employees using specified delay and wait parameters.

	calculateManyToManyRoute(vehicles: List<Vehicle>, employees: List<Employee>, maxDelay: int, waitTime: int): List<Route> – Computes routes for scenarios involving multiple vehicles and employees.
VroomClient	Class State - No persistent state (acts as an interface to the external VROOM API). Class Behavior - sendRoutingRequest(req: RoutingRequest): Route – Sends a routing request to the VROOM API and returns the computed route.
GoogleMapsClient	Class State - No persistent state (integrates with Google Maps services). Class Behavior - getDirections(src: Coordinates, dest: Coordinates): Route – Retrieves directions between two geographic coordinates. - getTrafficData(): TrafficData – Obtains real-time traffic information.
Route	Class State - routeId: String – Unique identifier for the route. - stops: List<Stop> – List of stops (pickup and drop-off points) along the route. - distance: double – Total distance covered by the route. - duration: double – Estimated duration for traversing the route. Class Behavior - display(): void – Outputs or renders the route details.
Vehicle	Class State - vehicleId: String – Unique identifier for the vehicle. - capacity: int – Maximum passenger capacity (moved from global constraints into the vehicle). - currentLocation: Coordinates – The current geographic location of the vehicle. Class Behavior

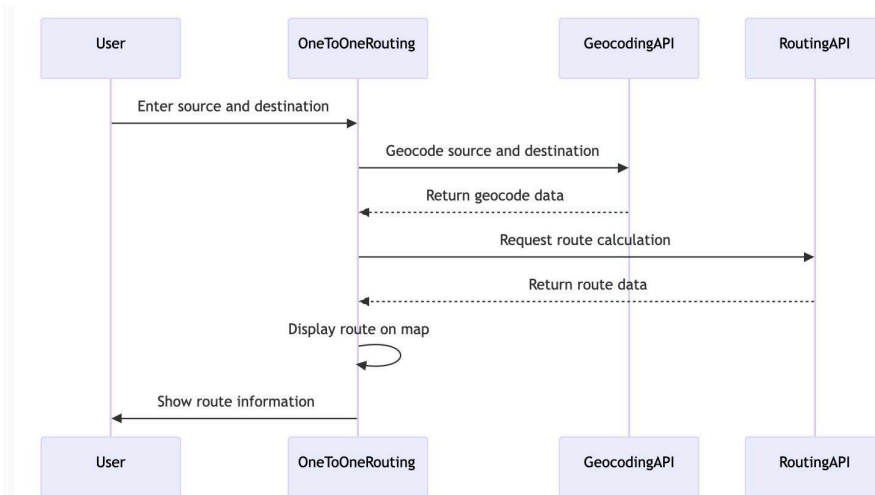
	• <code>updateLocation(loc: Coordinates): void</code> – Updates the vehicle's current location. • <code>assignRoute(route: Route): void</code> – Associates a computed route with the vehicle.
RealTimeTracker	Class State • Internally maintains a mapping of vehicle IDs to their live locations. Class Behavior • <code>updateVehicleLocation(vehicleId: String, loc: Coordinates): void</code> – Updates the live location of a vehicle. • <code>getVehicleLocation(vehicleId: String): Coordinates</code> – Retrieves the current location of a specific vehicle.
Map Visualiser	Class State • No persistent state (focused solely on rendering and updating maps). Class Behavior • <code>renderRoute(route: Route): void</code> – Renders the given route on a map interface. • <code>updateMap(route: Route): void</code> – Updates the map visualization when route changes occur.

Sequence Diagrams

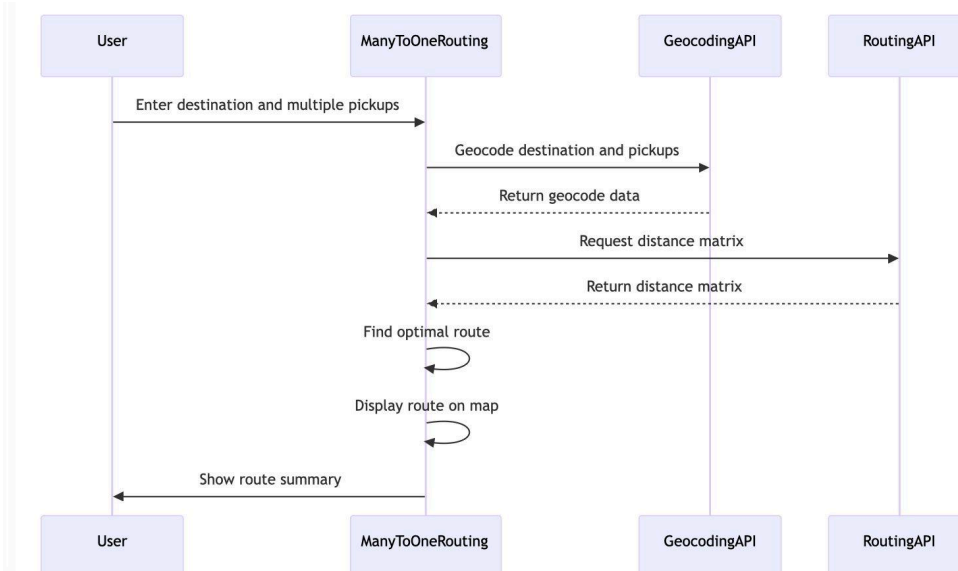
1. Geocode Employee Addresses



2. Create Optimized One-to-One Route



3. Create Optimized Many-to-One Route



Design Rationale

In a nutshell, the project can be broken down into 3 major steps :

1. **Routing** : Generate a path covering a given set of locations.
2. **Optimisation** : Identify the most optimal route based on distance, time or some other criteria.
3. **Constraints** [OPTIONAL] : Optimise route while considering constraints such as time windows, vehicle capacity, and real-time traffic conditions.

Each of these steps involves several underlying components, including, but not limited to, Road Network Graph Creation (data collection and preprocessing to build an accurate road network), One-to-One Routing, Many-to-One Routing and Many-to-Many Routing.

There are additional components to the project that, while not explicitly specified by the client, are essential for a seamless experience. These include a frontend user interface with an intuitive design, optimized performance, and minimal latency.

Our product design is driven by extensive market research and industry analysis. The approach evolves continuously as we identify challenges and improve solutions. Any changes to the methodology are made based on practical limitations, performance trade-offs, and the need for more efficient algorithms.

All of the major changes / updates are mentioned as below :

1. Initial Design

We chose OpenStreetMap (OSM) as our primary road network data source, integrated with OSRM (Open Source Routing Machine) for routing and optimization.

1a. Why OSM + OSRM?

Several factors influenced this decision. Some of the major ones are as follows :

- Open-Source & Cost-Effective : Unlike proprietary solutions, OSM and OSRM are free to use, making them ideal for a prototype.
- Flexible & Extensible : The open data format allows customization and integration with additional datasets if needed.
- Strong Community Support : Active contributions ensure continuous updates and improvements.
- Proven Industry Adoption : Many organizations use OSM/OSRM successfully for routing applications.

1b. Issues & Trade-offs with OSM

Despite its advantages, OSM comes with many challenges :

- Incomplete Road Coverage : Missing smaller roads can lead to suboptimal route calculations.
- Limited Location Recognition: Often requires precise address strings instead of natural-language place names.

We accepted these limitations for the prototype stage, with a plan to explore mitigation strategies later - such as enhancing data collection or leveraging machine learning to refine routing (out of scope for this phase).

1c. Why Not Google Maps API or Other Commercial Solutions?

Alternative solutions like Google Maps API were considered but ultimately rejected due to :

- Cost Barriers : Usage-based pricing could escalate quickly, making it impractical for large-scale routing.
- Data Limitations: While Google Maps excels in real-time traffic data, its routing API doesn't always provide the raw road network data flexibility needed for customization.

2. Towards Google API

While our initial evaluation favoured OpenStreetMap (OSM) plus Open Source Routing Machine (OSRM) library due to its open-source nature, flexibility, and cost-effectiveness; further testing revealed significant limitations in OSM's data quality - most notably, its failure to recognize the client's apartment address during a test run. This raised concerns about its suitability for our use case, particularly in handling real-world address searches.

Following detailed discussions on the pricing, performance and features, the client has asked to proceed with Google Maps API (free trial period) for a more accurate and reliable mapping solution.

2a. Why Google Maps API?

Several factors led to this change in design changes. The major ones are as follows :

- Superior Address Recognition : Unlike OSM, Google Maps API successfully handles precise address searches, improving routing accuracy.
- Robust Routing & Optimization : Provides real-time traffic data and optimized routing based on multiple factors.
- Ease of Integration : Well-documented APIs with seamless implementation options.

2b. Challenges & Trade-offs

- Cost Considerations : Google APIs operate on a pay-as-you-go model, and future scaling may require careful budget planning.
- Limited Customization: Unlike OSM, raw network data manipulation is restricted.

2c. Next Steps

The current implementation will leverage Google Maps API under the free trial period, allowing us to assess its performance in real-world scenarios. Once the trial ends, we will evaluate the cost implications and potential long-term feasibility before finalizing this as the permanent solution.